

Topologias de Equipe

Organizar equipes para fluxo rápido

Processos de Software — Unidade 3 (conteúdo final, parte 1/2)

Prof. Fernando · UFRN · 2026.1

De onde viemos

Já vimos medir (DORA), enxergar o fluxo (VSM) e achar o gargalo.

Descoberta recorrente	Implicação
O gargalo é estrutural	filas, dependências, espera por outra pessoa/time
Esperas vêm de passagens de bastão	passar trabalho de um grupo a outro custa tempo
Otimizar uma etapa local não resolve	o problema está na organização do trabalho

Subimos um nível: a forma como as **equipes** se organizam molda o fluxo — e até a arquitetura do software.

Lei de Conway (1968)

"Organizações que projetam sistemas produzem desenhos que são **cópias da estrutura de comunicação** dessas organizações." — Melvin Conway

Em uma frase: **a arquitetura do software espelha a organização que o constrói.**

```
3 times separados (frontend | backend | banco)
      ↓ (Lei de Conway)
arquitetura em 3 camadas acopladas, com passagens de bastão entre times
```

Não é fruto do acaso: é consequência de como a comunicação flui. Se dois módulos precisam conversar, os times por trás deles também precisam.

Conway na prática

A estrutura de comunicação vaza para dentro do código:

Organização	Tende a gerar
Times por camada técnica (UI, API, BD)	Acoplamento entre camadas; toda feature cruza 3 times
Times por função (devs, QA, ops separados)	Passagens de bastão e filas — o gargalo do VSM!
Times por fluxo de valor (feature ponta a ponta)	Menos passagens de bastão, entrega mais rápida

Lembrem do VSM: cada passagem de bastão entre times é uma **fila** em potencial. Menos passagens de bastão = menos espera.

Inverse Conway Maneuver

Se a organização molda a arquitetura, **desenhe a organização para induzir a arquitetura que você quer.**

Quer serviços independentes?

→ times pequenos e autônomos, donos de um serviço cada

Quer um produto coeso?

→ um time orientado ao fluxo de valor dono do fluxo inteiro

Em vez de lutar contra a Lei de Conway, use-a a favor: a estrutura de times é uma **alavanca de arquitetura.**

Carga cognitiva

Todo time tem um limite de **quanta complexidade** carrega bem.

Tipo de carga	Exemplo	O que fazer
Intrínseca	"como escrever um handler"	treinar, dominar o essencial
Extrínseca	"como configurar o deploy"	eliminar (plataforma, automação)
Pertinente	"qual a melhor modelagem do domínio"	proteger — é onde está o valor

Time sobrecarregado vira gargalo. A meta: limitar o escopo de cada time ao que **cabe** na sua carga cognitiva.

Pensar "time primeiro"

Team Topologies inverte a ordem: **primeiro o time, depois o trabalho.**

- Times **estáveis e de longa duração** (não montados/desfeitos por projeto)
- Cada time **dono** de um domínio ou fluxo
- O time tem uma **"interface"**: como os outros interagem com ele (repositório, docs, canais, formas de trabalho)

A unidade de entrega não é o indivíduo — é o **time**. Estabilidade do time é pré-requisito para fluxo.

Os 4 tipos de equipe

Team Topologies propõe **apenas quatro** tipos fundamentais:

Tipo	Papel em uma frase
Orientado ao fluxo de valor (stream-aligned)	Entrega um fluxo de valor ponta a ponta (o tipo principal)
Plataforma (platform)	Oferece uma plataforma interna de auto serviço que reduz carga cognitiva
Habilitador (enabling)	Ajuda outros times a adquirir capacidades que faltam
Subsistema complicado (complicated-subsystem)	Cuida de uma parte que exige conhecimento muito especializado

A maioria dos times deve ser **orientado ao fluxo de valor**. Os outros três existem para **apoiá-los**.

Equipe Orientada ao fluxo de valor

O time alinhado a um **fluxo de valor** — uma feature, produto ou jornada do usuário.

- Dono do fluxo de ponta a ponta: do código ao deploy ao monitoramento
- Autônomo: minimiza dependências de outros times
- É o tipo **primário** — todos os outros existem para reduzir sua carga

No projeto de vocês, a equipe **é** um time orientado ao fluxo de valor: entregam o produto inteiro. Os outros tipos surgem quando a organização cresce.

Equipe de Plataforma

Fornece uma **plataforma interna** (CI/CD, ambientes, observabilidade) como **auto serviço**.

Sem plataforma: cada time configura pipeline, deploy, monitoramento (carga extrínseca!)

Com plataforma: o time consome tudo pronto → foca no produto

A plataforma existe para **reduzir a carga cognitiva** dos times orientado ao fluxo de valor. Lembram da Sprint 2 (CI/CD, Docker)? Uma plataforma empacota isso para todos.

Equipe Habilitadora e de Subsistemas Complexos

Tipo	Quando existe	Exemplo
Habilitador	Um time orientado ao fluxo de valor precisa de uma capacidade que não tem	Especialistas em testes/segurança que ensinam o time por um período
Subsistemas Complexos	Uma parte exige conhecimento profundo e raro	Motor de cálculo, codec de vídeo, modelo de ML

Equipes habilitadoras **não fazem pelo time** — elas **habilitam** o time e depois saem do projeto. Equipes de subsistemas complexos evitam que todo mundo precise ser especialista naquela parte.

Os 3 modos de interação

Como dois times se relacionam — só três modos:

Modo	Como é	Quando usar
Colaboração	Trabalham juntos, alta troca	Descoberta, inovação, território novo (custa caro)
X-as-a-Service	Um consome o que o outro provê, com interface clara	Entrega previsível, baixo acoplamento
Facilitação	Um ajuda/mentora o outro	Equipe habilitadora apoiando uma equipe orientada ao fluxo de valor

Colaboração é poderosa, mas **cara** (muita comunicação). O objetivo de longo prazo costuma ser migrar para **X-as-a-Service** quando a interface amadurece.

Os modos evoluem

A interação não é fixa — muda com a maturidade:

Time novo precisa de algo da plataforma:

Colaboração (descobrir juntos a interface)

↓ quando estabiliza

X-as-a-Service (auto serviço, sem reuniões)

Colaboração eterna vira desperdício (reunião sem fim). Se dois times "colaboram" para sempre, talvez deveriam ser **um** time — ou a interface precisa virar serviço.

Sinais de problema (anti-padrões)

Sintoma	Provável causa
Toda feature passa por 3+ times	Times por camada, não por fluxo
Esperas longas por "outro time"	Passagens de bastão demais (gargalo no VSM)
Um time "faz tudo" e vive sobrecarregado	Carga cognitiva acima do limite
Times montados/desfeitos a cada projeto	Falta de estabilidade — sem dono claro

Conectem com o **VSM de vocês**: cada espera "aguardando outro time/aprovação" é sinal de uma fronteira de time mal desenhada.

No projeto de vocês

Mesmo com uma equipe só, dá para aplicar a lente:

- Vocês são um time **orientado ao fluxo de valor** (dono do fluxo inteiro)
- Onde houve **passagens de bastão internos**? (ex.: "só fulano faz deploy")
- Que **carga extrínseca** dava para eliminar com automação?
- Que **modo de interação** usaram com ferramentas/serviços externos?

Rende reflexão direta para a **Proposta de Melhoria** e para o vídeo: a organização do trabalho explica boa parte dos seus gargalos.

Checklist do conteúdo

- [] Sei enunciar a **Lei de Conway** e dar um exemplo
- [] Entendo a **Inverse Conway Maneuver**
- [] Sei os **4 tipos de equipe** e o papel de cada um
- [] Sei os **3 modos de interação** e quando usar cada
- [] Consigo ligar **passagens de bastão** do meu VSM a fronteiras de time

Topologias de Equipe entra como **reflexão** no vídeo/Proposta — não na prova (que já foi em 11/06).

Referências da aula

- Skelton & Pais — *Team Topologies* (cap. 1-3) — leitura **L15**
- Conway (1968) — *How Do Committees Invent?* — leitura **L16**
- Forsgren, Humble & Kim — *Accelerate* — arquitetura e times para desempenho
- teamtopologies.com — padrões e recursos